

# 깃허브와 주사위 게임

프로그래밍 입문  
2026964001 김동교  
제출일: 6월 12일

GitHub 주소: <https://github.com/sgdn12/Python>

sgdn12/Python

FPS N/A | GPU 0% | CPU 4% | 지연시간 N/A

https://github.com/sgdn12/Python

sgdn12 / Python

Type ↵ to search

+ - ⌂ ⌕ 📄 📧 🌐

<> Code ⌚ Issues 🔗 Pull requests 🛠 Agents ⌚ Actions 📁 Projects 📖 Wiki 🛡 Security and quality 📊 Insights ⚙ Settings

Python Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file T Add file <> Code

sgdn12 Update README.md dbcb30b · 4 minutes ago 44 Commits

|           |                  |                |
|-----------|------------------|----------------|
| 0306      | Update readme.md | 16 minutes ago |
| 0313      | Update readme.md | 12 minutes ago |
| 0320      | Update readme.md | 11 minutes ago |
| 0327      | Update readme.md | 10 minutes ago |
| 0403      | Create readme.md | 2 months ago   |
| 0410      | Update readme.md | 8 minutes ago  |
| 0417      | Update readme.md | 7 minutes ago  |
| 0508      | Update readme.md | 7 minutes ago  |
| 0515      | Update readme.md | 6 minutes ago  |
| README.md | Update README.md | 4 minutes ago  |

README

# Python

## 1. 파이썬 소개

컴퓨터는 하드웨어와 소프트웨어로 구성되어 사람이 작성한 명령어 목록인 '프로그램'을 통해 인공지능, 주식 자동매매 등 다양한

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

sgdn12

Languages

Python 100.0%

Suggested workflows

Based on your tech stack

# 1주차: 파이썬 기초

readme.md

## 1. 파이썬 소개

컴퓨터는 하드웨어와 소프트웨어로 구성되어 사람이 작성한 명령어 목록인 '프로그램'을 통해 인공지능, 주식 자동매매 등 다양한 분야에서 반복적이고 지루한 작업을 빠르고 정확하게 처리합니다. 컴퓨터는 사람의 언어를 직접 이해하지 못하기 때문에 프로그래밍 언어가 필요한데, 사람이 파이썬 같은 언어로 코드를 작성하면 컴파일러나 인터프리터가 이를 컴퓨터가 이해할 수 있는 기계어로 번역해 줍니다.

특히 파이썬은 문법이 간결하고 결과를 바로 확인할 수 있는 인터프리터 언어라 초보자가 배우기 좋습니다. 또한 라이브러리가 풍부하여 데이터 분석, 웹 개발 등 여러 분야에 활용되며, 코드와 설명을 함께 작성할 수 있는 주피터 노트북이나 Colab 환경에서 주로 사용됩니다. 소스 파일의 확장자는 .py입니다.

파이썬의 기초 문법으로는 화면에 문자열이나 계산 결과를 출력하는 `print()` 함수가 있습니다. 이때 문자열은 반드시 큰따옴표나 작은따옴표로 감싸야 하며, 따옴표가 없으면 오류가 발생합니다. 또한 파이썬은 대소문자를 엄격하게 구분하기 때문에 `print`를 `Print`로 잘못 적거나 따옴표, 괄호 등을 누락하지 않도록 주의해야 합니다.

예시로는

```
print("Hello Python")
```

## Commits

History for Python / 0306 on main

🔍 All users ▾ 📅 All time ▾

⌵ Commits on Jun 10, 2026

Update readme.md

● sgdn12 authored 2 hours ago

Verified

8f22530

📄 🗑️ <>

Add introduction section to readme

● sgdn12 authored 2 hours ago

Verified

68396f8

📄 🗑️ <>

⌵ Commits on Mar 12, 2026

Add files via upload

● sgdn12 authored on Mar 12

Verified

b2b35c8

📄 🗑️ <>

Add files via upload

● sgdn12 authored on Mar 12

Verified

59ef7cd

📄 🗑️ <>

⌵ Commits on Mar 6, 2026

Create readme.md

● sgdn12 authored on Mar 6

Verified

92002b7

📄 🗑️ <>

⌵ End of commit history for this file



## Lab: 간단한 계산

```
2+3= 5
2-3= -1
2*3= 6
2/3= 0.6666666666666666
```

```
>>> print("2+3=", 2+3)
2+3= 5
>>> print("2-3=", 2-3)
2-3= -1
>>> print("2*3=", 2*3)
2*3= 6
>>> print("2/3=", 2/3)
2/3= 0.6666666666666666
```

## Lab: 오류를 처리해보자.

```
print("안녕하세요?)
Print("이번 코드에는 많은 오류가 있대네요")
print("제가 다 고쳐 보겠습니다.")
```

```
File "D:\모두의 파이썬/sources/chap01/hello.py", line 1
print(안녕하세요?)
      ^
SyntaxError: invalid syntax
```

```
>>> print("안녕하세요?")
안녕하세요?
>>> print("이번 코드에는 많은 오류가 있대네요")
이번 코드에는 많은 오류가 있대네요
>>> print("제가 다 고쳐 보겠습니다.")
제가 다 고쳐 보겠습니다.
```

```
3 + 1 = 3   3 * 2 = 6   3 + 3 = 9
```

```
>>> 3 + 1
3
>>> 3 * 2
6
>>> 3 + 3
9
```

## 이번 장에서 배운 것

- 프로그램은 컴퓨터에 내리는 명령어로 이루어지는 작업 지시서이다.
- 고급 언어는 컴퓨터가 이해할 수 있는 언어이다.
- 다양한 종류의 프로그래밍 언어가 있고 파이썬도 프로그래밍 언어의 일종이다.
- 파이썬으로 프로그램을 작성하기 위한 개발 환경인 **아나콘다**는 <https://www.anaconda.com/distribution/> 웹사이트에서 다운로드해서 설치할 수 있다.
- IPython 콘솔에서는 프롬프트 다음에 코드를 입력하고 **Enter**를 누르면 코드가 실행된다.
- 산술 계산을 하는 파이썬 연산자에는 +, -, \*, /가 있다.
- print()는 화면에 문자열이나 계산 결과를 출력할 때 사용하는 함수이다.
- 스크립트 모드를 사용하면 코드를 파일에 저장하였다가 한꺼번에 실행할 수 있다.

## Lab: print() 함수 실습

파이썬에 오신 것을 환영합니다.  
파이썬은 쉽습니다.  
파이썬으로 빅데이터, 인공지능 프로그램을 작성할 수 있습니다.

```
>>> print("파이썬에 오신 것을 환영합니다.")
파이썬에 오신 것을 환영합니다.
>>> print("파이썬은 쉽습니다.")
파이썬은 쉽습니다.
>>> print("파이썬으로 빅데이터, 인공지능 프로그램을 작성할 수 있습니다.")
파이썬으로 빅데이터, 인공지능 프로그램을 작성할 수 있습니다.
```

## 중간점검

1. 컴퓨터의 장점은 무엇인가?  
-컴퓨터의 장점은 빠른 계산 속도, 정확성, 많은 양의 데이터 처리 능력, 반복 작업이다.
2. 왜 계산기는 컴퓨터라고 할 수 없는가?  
-계산기는 정해진 계산 기능만 수행할 수 있다. 하지만 컴퓨터는 프로그램을 통해 다양한 일을 수행할 수 있다.
3. 프로그램 안에는 무엇이 들어 있는가?  
-프로그램에는 컴퓨터가 해야 할 일을 순서대로 적은 명령어가 들어 있다.
4. 컴퓨터의 장점과 인간의 장점은 무엇인가?  
-컴퓨터의 장점: 빠르고 정확한 계산과 반복 작업을 잘한다.  
인간의 장점: 창의력 사고와 판단력, 문제 해결 능력이 뛰어나다.
5. 왜 우리는 프로그래밍을 배우야 하는가?  
-프로그래밍을 배우면 컴퓨터에게 원하는 일을 시킬 수 있고, 문제 해결 능력과 논리적 사고를 키울 수 있다.
6. 한글도 출력될까? 이번에도 마우스를 올바르게 입력하여야 한다. "안녕하세요?"를 화면에 출력하여 보자.  

```
>>> print("안녕하세요?")
안녕하세요?
```
7. "programming에 입문하신 것을 축하드립니다."를 출력하여 보자.  

```
>>> print("programming에 입문하신 것을 축하드립니다.")
programming에 입문하신 것을 축하드립니다.
```
8. "생일 축하!!"를 10번 출력하는 명령문을 만들어보자. 문자열 반복을 사용한다.  

```
>> print("생일 축하!!" * 10)
생일 축하!!생일 축하!!생일 축하!!생일 축하!!생일 축하!!생일 축하!!생일 축하!!생일 축하!!생일 축하!!생일 축하!!
```
9. 다음과 같은 명령문을 실행하면 오류가 발생한다. 원인을 알아보자.>>> print("Hello")  
-print("Hello")에서는 앞의 큰따옴표(")는 있지만 뒤의 큰따옴표(")가 없어 오류가 발생한다.
10. 스파이더는 두 가지 모드로 사용할 수 있다. 두 가지 모드에 대하여 설명해보자.  
-대화 모드: 한 줄씩 즉시 실행한다.  
스크립트 모드: 여러 줄 작성 후 파일로 저장하여 한 번에 실행한다.
11. 파이썬으로 프로그램을 작성할 때 대문자와 소문자를 구분할까?  
-파이썬은 대문자와 소문자를 구분하기 때문에 print를 Print로 적으면 구문 오류가 발생하게 된다.
12. 파이썬 소스 파일의 확장자는 무엇인가?  
-파이썬의 소스 파일의 확장자는 .py 이다.
13. 다음과 같이 3단 구구단의 일부를 출력하는 프로그램을 작성해보자.

# 2주차: 변수와 수식

readme.md

## 2. 변수와 수식

변수는 데이터를 저장하기 위해 메모리에 이름을 붙여둔 공간이며, 파이썬에서는 값을 대입하는 순간 자동으로 생성됩니다. 변수의 값은 언제든지 새로운 값으로 바뀔 수 있습니다. 계산을 위한 산술 연산자에는 사칙연산 외에도 거듭제곱(), 몫(/), 나머지(%) 등이 있으며, input() 함수로 입력받은 데이터는 문자열이 되므로 계산하려면 int() 등을 통해 숫자로 형 변환을 해야 합니다.

예시

```
num = int(input("숫자를 입력하세요: "))
result = num * 2
print("2를 곱한 결과:", result)
```

Commits

History for Python / 0313 on [ex35c12](#)

Commits on Jun 10, 2026

Update readme.md

sgin12 authored 2 hours ago

Verified ex35c12

Update readme.md

sgin12 authored 2 hours ago

Verified 8b4c4e6

Commits on Mar 18, 2026

Add files via upload

sgin12 authored on Mar 18

Verified 649b032

Delete 0313/0306-2026964001 (1).hwp

sgin12 authored on Mar 18

Verified 0b43b08

Add files via upload

sgin12 authored on Mar 18

Verified ex379d9

Commits on Mar 13, 2026

Add files via upload

sgin12 authored on Mar 13

Verified 4ca08d8

Add files via upload

sgin12 authored on Mar 13

Verified 065efed

Add files via upload

sgin12 authored on Mar 13

Verified 0b263d1

Add files via upload

sgin12 authored on Mar 13

Verified 4520616

Commits on Mar 6, 2026

Create readme.md

sgin12 authored on Mar 6

Verified a379b7b

9. 변수 k에 300을 저장하는 문장을 작성하여 보자.

- k = 300

10. 다음 프로그램은 사각형의 면적을 계산한다. 코드에 주석을 붙여보자.

```
1 # 사각형의 면적 계산 프로그램
2 #
3 # 이 프로그램은 사각형의 넓이를 구하는 코드이다.
4 #
5 width = 10 # 폭(가로)의 길이
6 height = 5  # 높이(세로)의 길이
7 print("사각형의 넓이는: ", width*height)
```

11. 복합 대입 연산자 x += y의 의미를 설명하라.

x에다가 x+y를 대입

12. 10%의 값은 무엇인가?

- 4

13. 나눗셈 연산인 10//6의 값은 얼마인가?

- 1

14. 다음의 할당문에서 무엇이 잘못되었는가?

3 = x

- 대입할 때는 변수와 이름을 왼쪽에 적어야 한다.

15. 다음의 할당문이 실행된 후의 변수 a, b, c의 값은?

a = b = c = 100

- a = 100, b = 100, c = 100

16. 문자열 "100"을 변수 100으로 변환하는 명령문을 작성하라.

- s = 100

- result = int(s)

17. 변수 100을 문자열 "100"으로 변환하는 명령문을 작성하라.

- n = 100

- result = str(n)

18. 다음과 같이 3개의 문자열이 3개의 변수에 저장되어 있다. 첫글자만을 따서 "BTW"라고 만들고 싶다. 어떻게 하면 되는가?

a = "By"

b = "Th"

c = "Way"

19. 사용자로부터 2개의 정수를 받아서 사칙연산(+, -, \*, /)을 한 후에 결과를 출력하는 프로그램을 작성해보자.

```
C:\Users> cd C:\Python\0313 > python 1.py
1 num1 = int(input("첫 번째 정수를 입력하세요: "))
2 num2 = int(input("두 번째 정수를 입력하세요: "))
3
4 print(num1, "+", num2, "=", num1 + num2)
5 print(num1, "-", num2, "=", num1 - num2)
6 print(num1, "*", num2, "=", num1 * num2)
7 print(num1, "/", num2, "=", num1 / num2)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

첫 번째 정수를 입력하세요: 25  
두 번째 정수를 입력하세요: 55  
25 + 55 = 80  
25 - 55 = -30  
25 \* 55 = 1375  
25 / 55 = 0.45454545454545453

20. 사용자의 이름을 물어보고 이어서 2개의 정수를 받아서 덧셈을 한 후에 결과를 출력하는 다음과 같은 프로그램을 작성해보자.

```
1 # 사용자 이름과 두 정수를 받아서 덧셈하는 프로그램
2 #
3 # 이 프로그램은 사용자의 이름을 물어보고, 두 개의 정수를 받아서 그 합을 출력하는 코드이다.
4 #
5 name = input("이름을 입력하세요: ")
6 print("안녕하세요, " + name + "님!")
7
8 # 두 개의 정수를 입력받아서 합을 구한다.
9 num1 = int(input("첫 번째 정수를 입력하세요: "))
10 num2 = int(input("두 번째 정수를 입력하세요: "))
11
12 print(num1, "+", num2, "=", num1 + num2)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

이름을 입력하세요: 홍길동  
파이썬에 오신 것을 환영합니다.  
두 번째 정수를 입력하세요: 400  
첫 번째 정수를 입력하세요: 300  
300 과 400 의 합은 700입니다.

중간점검

1. 변수를 다시 설명해보자.

- 컴퓨터의 메모리 공간에 이름을 붙여둔 것

2. 밑변이 10이고 높이가 10인 삼각형의 면적을 계산하는 프로그램을 작성해 보자.

```
1 # 삼각형의 면적 계산 프로그램
2 #
3 # 이 프로그램은 삼각형의 넓이를 구하는 코드이다.
4 #
5 base = 10 # 밑변의 길이
6 height = 10 # 높이의 길이
7 print("삼각형의 면적은: ", base*height/2)
```

3. 변수 이름을 만들 때 지켜야 하는 규칙은 무엇인가?

- 숫자가 앞에 오면 안되고, 중간에 공백이 있으면 안되고, 예약어(변수 이름으로 사용하면 안되며, 키워드)를 변수 이름으로 사용하지 않는다.

4. 변수 이름의 첫 번째 글자로 허용되는 것은 무엇인가?

- 영어와 -가 가능하다.

5. 파이썬에서 중요한 의미를 가지고 있는 단어들을 무엇이라고 하는가?

- True, False, and, or, not, if, else, while, for, break, continue

6. 파이썬에는 어떤 자료형이 있는가? 3가지만 말해보자.

- 정수형, 실수형, 문자형

7. 파이썬 변수에 정수를 저장했다가 실수를 저장하는 것도 가능한가? 그 이유는 무엇인가?

- 가능하다. 동적타입으로 인해 파이썬은 변수를 설정할 때 실시간으로 자료형이 바뀐다.

8. 변수의 자료형을 알려면 어떤 함수를 사용하는가?

- type



 Lab: 별까지의 거리 계산하기 

```

3 c:\Users\USER> C:\Users\USER> python -i %*
4 speed = 300000.0
5 distance = 4000000000000.0
6 secs = distance / speed
7 light_year = secs / (60*60*24*365.0)
8
9 print(light_year, "년")
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
102
```

👩🏫 Lab: 원기둥의 부피 계산

```
1 radius = 5
2 height = 10
3 V = 3.14 * radius * radius * height
4 print("반지름=", radius, "높이=", height, "원기둥의 부피 =", V)
```

 Lab: 대화하는 프로그램 만들기

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 print("안녕하세요?")
2 name = input("이름이 어떻게 되시나요? ")
3 print("만나서 반갑습니다. " + name + "씨")
4 print("이름의 길이는 다음과 같군요: ", len(name))
5 age = int(input("나이가 어떻게 되나요? "))
6 next_year_age = age + 1
7 print("내년이면", next_year_age, "이 되시는군요.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

itled-1.py
안녕하세요?
이름이 어떻게 되시나요? 김동교
만나서 반갑습니다. 김동교씨
이름의 길이는 다음과 같군요: 3
나이가 어떻게 되나요? 20
내년이면 21 이 되시는군요.
PS C:\Users\USER>
```

## Lab: BMI 계산하기

```
C:\Users\USER> OneDrive> 문서 > Untitled-1.py > ...
1 weight = float(input("몸무게를 kg 단위로 입력하십시오: "))
2 height = float(input("키를 미터 단위로 입력하십시오: "))
3 bmi = (weight / (height**2))
4 print("당신의 BMI=", bmi)
```

 Lab: 복리 계산

```
C:\Users\USER> C:\Users\USER> python3 -i
1 init_money = 24
2 interest = 0.06
3 years = 182
4 print(init_money*(1+interest)**years)
```

 Lab: 로봇 기자 만들기

```

1 stadium = input("경기장명은 무엇입니까? ")
2 winner = input("이긴 팀은 어디입니까? ")
3 loser = input("진 팀은 어디입니까? ")
4 vip = input("우승금은 누릅니까? ")
5 score = input("스코어는 몇대 얼마입니까? ")
6 print("")
7 print("-----")
8 print("오늘, stadium, "에서 야구 경기가 열렸습니다.")
9 print("winner, "과, loser, "은 치열한 공방전을 펼쳤습니다.")
10 print("vip, "이 엄청난 금액을 받았습니다.")
11 print("결과, winner, "과 loser, "은, score, "로 이겼습니다.")
12 print("-----")

```

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\USER> C:\Users\USER\AppData\Local\Programs\Python\Python114\python.exe c:\Users\USER\OneDrive\문서\11-11-19  
경기장명은 어디입니까?  
이긴 팀은 어디입니까?  
진 팀은 어디입니까?  
우승금은 누릅니까?  
스코어는 몇대 얼마입니까?

오늘 사직 에서 야구 경기가 열렸습니다.  
롯데 와 한화 은 치열한 공방전을 펼쳤습니다.  
한화 이 승리를 확정했습니다.  
승금 돈에 가 한화 4 대 2 로 이겼습니다.

 Lab: 구의 부피 계산하기

```
C:\> Users > Users > OneDrive > 임시 > Untitled-1.py > volume

1  r = float(input("반지름을 입력하시오: "))
2  volume = (4.0/3.0) * 3.141592 * r**3
3  print("구의 부피-", volume)
```

## Lab: 자동판매기 프로그램

```

1 itemPrice = int(input("请输入商品价格:"))
2
3 note = int(input("请输入纸币数:"))
4
5 coin100 = int(input("请输入100元纸币数:"))
6
7 coin100 = int(input("请输入100元纸币数:"))
8
9 change = note*1000 - coin500*1000 - itemPrice
10
11 coin100 = change//500
12
13 change = change%500
14
15 coin50 = change//100
16
17 coin50 = change//100
18
19 change = change%100
20
21 coin10 = change//10
22
23 change = change%10
24
25 coin1 = change//1
26
27 print("100元:", coin100, "100元:", coin100, "10元:", coin10, "1元:", coin1)

```

# 3주차: 조건문

readme.md

## 3. 조건문

조건문은 주어진 조건에 따라 프로그램의 실행 흐름을 바꾸는 제어문입니다. if, elif, else 구조를 사용하며, 조건식에는 대소 비교를 하는 관계 연산자와 여러 조건을 묶어주는 논리 연산자(and, or, not)가 사용됩니다. 파이썬에서는 조건문 뒤에 실행할 명령문들을 반드시 동일한 깊이로 들여쓰기해야 같은 블록으로 인식됩니다.

예시

```
score = 85
if score >= 60:
    print("합격입니다.")
else:
    print("불합격입니다.")
```

1. 프로그램에 사용되는 3가지의 제어구조는 어떤 것들인가?  
- 순차구조, 반복구조, 선택구조
2. 제어문은 조건문과 반복문으로 나누어진다.
3. 다음의 조건에 해당하는 논리 연산식을 만들어 보시오. 변수는 적절하게 선언되어 있다고 가정한다.  
"나이는 25살 이상, 연봉은 3500만원 이상"  
- age >= 25 && salary >= 35000000
4. 수직 not True의 값은?  
- False

### Lab: 산술 퀴즈 프로그램

```
1 import random
2 x = random.randint(1,100)
3 y = random.randint(1,100)
4 answer = int(input(f"({x})+({y}) = "))
5 flag = (answer == (x+y))
6 print(flag)
7 number = int(input(f"({x})-(y) = "))
8 flag = (answer == (x-y))
9 print(flag)
```

```
python34/python.exe C:/Users/USER/OneDrive/문서/Untitled-1.py
50+50 = 100
True
50-50 = 10
True
```

### Commits

| History for Python / 0320 on <a href="#">wale</a> |                             | All users  | All time |
|---------------------------------------------------|-----------------------------|------------|----------|
| Commits on Jun 10, 2026                           |                             |            |          |
| Update readme.md                                  | sgdn12 authored 2 hours ago | (Verified) | 4259caf  |
| Commits on Mar 26, 2026                           |                             |            |          |
| Add files via upload                              | sgdn12 authored on Mar 26   | (Verified) | 150ba3e  |
| Delete 0320/list.py                               | sgdn12 authored on Mar 26   | (Verified) | 387badb  |
| Commits on Mar 22, 2026                           |                             |            |          |
| Add files via upload                              | sgdn12 authored on Mar 22   | (Verified) | 1b82335  |
| Commits on Mar 20, 2026                           |                             |            |          |
| Add files via upload                              | sgdn12 authored on Mar 20   | (Verified) | 64534ce  |
| Create readme.md                                  | sgdn12 authored on Mar 20   | (Verified) | 322395c  |
| Delete 0320                                       | sgdn12 authored on Mar 20   | (Verified) | 2d7b2b8  |
| Create 0320                                       | sgdn12 authored on Mar 20   | (Verified) | 970bc7   |

### Lab: 산술 퀴즈 프로그램

```
1 number = int(input("정수를 입력하십시오: "))
2 if number % 2 == 0:
3     print("짝수입니다.")
4 else:
5     print("홀수입니다.")
```

```
PS C:\Users\USER> python34/python.exe C:/Users/USER/OneDrive/문서/Untitled-1.py
정수를 입력하십시오: 21
홀수입니다.
```

```
1 number = int(input("정수를 입력하십시오: "))
2 if number >= 60:
3     print("합격입니다.")
4 else:
5     print("불합격입니다.")
```

```
python34/python.exe C:/Users/USER/OneDrive/문서/Untitled-1.py
정수를 입력하십시오: 60
합격입니다.
```

## Lab: 8 내식플

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 import random
2 print("<영문의 매직볼은 오늘의 문제를 출력합니다.>")
3 answer = random.randint(1, 2)
4 if answer == 1:
5     print("<확실한 이유입니다.>")
6 else:
7     print("<확실한 이유 없습니다.>")
8 if answer == 2:
9     print("<아니요.>")
10 else:
11     print("<네.>")
12 print("<이제 영문에는 no입니다.>")
13 else:
14     print("<다시 질문해주세요.>")
```

## Lab: 사용자 입력 검증하기

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 print("<메뉴 1번: 치즈 버거\n 메뉴 2번: 치킨 버거\n 메뉴 3번: 불고기>")
2 print("=====")
3 selection = int(input("<메뉴를 선택하세요:>"))
4 if selection >= 1 and selection <= 3:
5     print("<메뉴>", selection)
6 else:
7     print("<잘못 입력하셨습니다.>")
8
```

## Lab: 동전 던지기 게임

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 import random
2 print("<동전 던지기 게임을 시작합니다.>")
3 coin = random.randrange(2)
4 if coin == 0:
5     print("<앞면입니다.>")
6 else:
7     print("<뒷면입니다.>")
8 print("<게임이 종료되었습니다.>")
```

## Lab: 리히터 규모

```
1 scale = float(input("<리히터 규모를 입력하십시오:>"))
2 if scale >= 8.0:
3     print("<대부분의 구조물이 파괴됩니다.>")
4 elif scale >= 7.0:
5     print("<지표면에 균열이 발생합니다.>")
6 elif scale >= 6.0:
7     print("<일반적인 건물에 큰 피해가 있습니다.>")
8 elif scale >= 5.0:
9     print("<물건들이 흔들리거나 떨어집니다.>")
10 else:
11     print("<지진계에 의해서만 탐지 가능합니다.>")
```

## Lab: 세일 가격 계산

```
1 rate = 0.85
2 print("<10%에서 사은품을 받아주세요.>")
3 else:
4     rate = 0.90
5 dis_price = rate * price
6 print("<할인된 상품의 가격:>", dis_price)
```

## Lab: 물의 상태 출력하기

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 temp = int(input("<온도를 입력하십시오:>"))
2 if temp <= 0:
3     print("<물의 상태는 고체입니다.>")
4 elif temp > 0 and temp < 100:
5     print("<물의 상태는 액체입니다.>")
6 else:
7     print("<물의 상태는 기체입니다.>")
```

## Lab: 축구게임

```
C:\Users\USER> Downloads > import random.py > ...
1 import random
2 computer_choice = random.randint(1, 3)
3 user_choice = int(input("<컴퓨터를 선택하십시오(1: 공격, 2: 수비, 3: 오프)>"))
4 if computer_choice == user_choice:
5     print("<무승부입니다.>")
6 else:
7     print("<게임이 종료되었습니다.>")
```

## Lab: 올바른 삼각형 구분

```
C:\Users\USER> Downloads > import random.py > ...
1 a = int(input("<삼각형의 한 변을 입력하십시오:>"))
2 b = int(input("<삼각형의 한 변을 입력하십시오:>"))
3 c = int(input("<삼각형의 한 변을 입력하십시오:>"))
4 if (a+b) > c and (b+c) > a and (a+c) > b:
5     print("<올바른 삼각형>")
6 else:
7     print("<올바르지 않은 삼각형>")
```



# 4주차: 반복문

readme.md

## 4. 반복문

반복문은 동일하거나 유사한 작업을 여러 번 수행할 때 사용하며, 컴퓨터의 빠른 처리 능력을 극대화하는 문법입니다. 횟수가 정해진 반복에는 숫자 범위를 만들어주는 range() 함수와 for문을 주로 사용하고, 특정 조건이 참인 동안 계속 반복할 때는 while문을 사용합니다. while문을 쓸 때는 조건이 영원히 참이 되어 멈추지 않는 '무한 루프'에 빠지지 않도록 종료 조건을 잘 제어해야 합니다.

예시

```
for i in range(3):  
    print("방문을 환영합니다!")
```

## Commits

History for Python / 0327 on **main**

All users All time

Commits on Jun 10, 2026

### Update readme.md

sgdn12 authored 2 hours ago

Verified c09ae4b

Commits on Apr 2, 2026

### Add files via upload

sgdn12 authored on Apr 2

Verified 3fa7466

### Add files via upload

sgdn12 authored on Apr 2

Verified fd66768

Commits on Mar 27, 2026

### Create readme.md

sgdn12 authored on Mar 27

Verified fd13012

End of commit history for this file

중간점점

1. 프로그램에 반복 구조가 필요한 이유는 무엇인가?  
- 코드를 효율적으로 사용하고 간결하게 하기 위해 필요하다.
2. 반복 구조에는 횟수반복과. 조건반복이 있다.
3. 조건 반복과 횟수 반복을 설명해보자.  
-조건 반복은 특정한 조건이 만족되는 동안에 계속 반복한다.  
-횟수 반복은 반복하기 전에 반복 횟수를 미리 아는 경우에 사용한다.
4. [ 1, 2, 3, 4, 5 ]를 저장하는 리스트 myList를 생성해보자.

```
1 myList = [1, 2, 3, 4, 5]  
2 print(myList)
```

- myList의 첫 번째 항목을 0으로 변경해보자.

```
1 myList = [1, 2, 3, 4, 5]  
2 myList[0] = 0  
3 print(myList)
```

- myList의 마지막 항목을 9로 변경해보자.

```
1 myList = [1, 2, 3, 4, 5]  
2 myList[0] = 0  
3 myList[4] = 9  
4 print(myList)
```

- 7. 다음 코드의 출력을 쓰시오.

```
for i in [9, 8, 7, 6, 5]:  
    print("i=", i)  
i=9  
i=8  
i=7  
i=6  
i=5
```

- 8. 다음 코드의 출력을 쓰시오.

```
for i in range(1, 5, 1):  
    print(2*i)  
  
2  
4  
6  
8
```

- 9. 다음 코드의 출력을 쓰시오.

```
for i in range(10, 0, -2):  
    print("Student" + str(i))  
student10  
student8  
student6  
student4  
student2
```

- 10. if 문과 while 문을 비교하여 보라. 조건식이 같다면 어떻게 동작하는가?

-조건식이 같은 경우 if문은 조건식이 참이면 if코드 안의 코드를 한번 실행. 거짓이면 if코드 안의 코드를 실행하지 않는다. 그리고 elif의 조건식을 검사하여 불로 안의 코드를 실행하고, 종의 방식으로 동작한다. while문은 조건식이 참인 경우에는 끊임없이 반복하고 거짓인 경우에는 멈춘다.

- 11. for 문과 while 문을 비교해보자. 어떤 경우에 for 문을 사용하는 것이 좋은가?

또 어떤 경우에 while 문을 사용하는 것이 좋은가?

for 문을 사용할 때는 반복 횟수가 범위가 정해져있을 때 사용하기 좋고 while 문은 횟수가 명확하지 않을 때 사용하기 좋다.

- 12. 다음 코드의 출력을 쓰시오.

```
for i in range(3):  
    for j in range(3):  
        print(i, "곱하기", j, "은", i*j)
```

0 곱하기 0 은 0  
0 곱하기 1 은 0  
0 곱하기 2 은 0  
1 곱하기 0 은 0  
1 곱하기 1 은 1  
1 곱하기 2 은 2  
2 곱하기 0 은 0  
2 곱하기 1 은 2  
2 곱하기 2 은 4

## Lab: 팩토리얼 계산하기

for문을 이용하여서 팩토리얼을 계산해보자.

정수를 입력하시오: 10  
10!은 3628800이다.

```
1 n = int(input("정수를 입력하시오: "))
2 fact = 1
3 for i in range(1, n+1):
4     fact = fact * i
5 print(n, "!은", fact, "이다.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
s:\import random.py
정수를 입력하시오: 10
10 !은 3628800 이다.
PS C:\Users\USER>
```

## 도전문제

1. 팩토리얼을 거꾸로 계산해보자.

$n! = n \times (n-1) \times \dots \times 2 \times 1$

앞의 프로그램을 어떻게 수정하여야 하는가?

```
C:\Users\USER> Downloads > import random.py >
1 n = int(input("정수를 입력하시오: "))
2 fact = 1
3 print(n, end="! = ")
4 for i in range(n, 1, -1):
5     print(i, end=" x ")
6     fact = fact * i
7 print("1", fact, end="이다.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python34\python.exe
s:\import random.py
정수를 입력하시오: 6
6! = 6 x 5 x 4 x 3 x 2 x 1 = 720이다.
PS C:\Users\USER>
```

## Lab: 초등생을 위한 산수 문제 발생기

```
1 import random
2 flag = True
3 while flag:
4     x = random.randint(1, 100)
5     y = random.randint(1, 100)
6     answer = int(input("(x) + (y) = "))
7     if answer == x + y:
8         print("잘했어요!!")
9     else:
10        print("틀렸어요. 하지만 다음번에는 잘할 수 있죠?")
11        flag = False
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
23 + 18 = 41
잘했어요!!
75 + 95 = 120
틀렸어요. 하지만 다음번에는 잘할 수 있죠?
```

## Lab: 로그인 프로그램

• 사용자가 암호를 입력하고 프로그램에서 암호가 맞는 지를 체크한다고 하자.

```
1 password = ""
2 while password != "pythonisfun":
3     password = input("암호를 입력하시오: ")
4 print("로그인 성공")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
s:\import random.py
암호를 입력하시오: 12121212
암호를 입력하시오: 232131
암호를 입력하시오: pythonisfun
로그인 성공
```

## Lab: 구구단 출력

```
1 dan = int(input("원하는 단은: "))
2 i = 1
3 while i <= 9:
4     print("%d*%d=%d" % (dan, i, dan*i))
5     i = i + 1
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

원하는 단은: 8  
8\*1=8  
8\*2=16  
8\*3=24  
8\*4=32  
8\*5=40  
8\*6=48  
8\*7=56  
8\*8=64  
8\*9=72

## Lab: 숫자 맞추기 게임

이 예제는 프로그램이 가지고 있는 정수를 사용자가 알아맞히는 게임이다. 사용자가 답을 제시하면 프로그램은 자신이 저장한 정수와 비교하여 제시된 정수가 더 높은지 낮은지만을 알려준다.

```
1 import random
2 guess = 0
3 answer = random.randint(1, 100)
4 print("1부터 100 사이의 숫자를 맞추세요")
5 while guess != answer:
6     guess = int(input("숫자를 입력하시오: "))
7     tries = tries + 1
8     if guess < answer:
9         print("너무 작음!")
10    elif guess > answer:
11        print("너무 높음!")
12
13 if guess == answer:
14    print("축하합니다. 시도횟수=", tries)
15 else:
16    print("정답은 ", answer)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
s:\import random.py
1부터 100 사이의 숫자를 맞추세요
숫자를 입력하시오: 66
너무 높음!
숫자를 입력하시오: 33
너무 높음!
숫자를 입력하시오: 55
너무 높음!
숫자를 입력하시오: 44
너무 높음!
숫자를 입력하시오: 68
너무 높음!
숫자를 입력하시오: 63
너무 높음!
숫자를 입력하시오: 53
축하합니다. 시도횟수= 7
```

## Lab: 방정식의 해 구하기

```
1 COUNT = 100
2 START = 1.0
3 END = 2.0
4 for i in range(COUNT):
5     x = START + 1*((END-START)/COUNT)
6     f = (x**2 - x - 1)
7     if abs(f-0.0) < 0.01:
8         print("방정식의 해는 ", x)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
s:\import random.py
방정식의 해는 1.62
PS C:\Users\USER>
```

(1) if abs(f-0.0) < 0.0001 : 으로 변경하여 보자. 또 COUNT 상수의 값을 100000으로 설정하여 보자. 어떤 일이 발생하는가?

```
1 COUNT = 100000
2 START = 1.0
3 END = 2.0
4 for i in range(COUNT):
5     x = START + 1*((END-START)/COUNT)
6     f = (x**2 - x - 1)
7     if abs(f-0.0) < 0.0001:
8         print("방정식의 해는 ", x)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python34\python.exe "C:\Users\USER\download
s:\import random.py
방정식의 해는 1.62799
방정식의 해는 1.628
방정식의 해는 1.62801
방정식의 해는 1.62801
방정식의 해는 1.62801
방정식의 해는 1.6280100000000001
방정식의 해는 1.6280100000000002
방정식의 해는 1.62801
방정식의 해는 1.62807
```

(2) 이분법(bisection) 방법을 인터넷에서 조사해보고 이분법을 구현해보자.

-이분법이란?

- (1) 방법: 범위를 정하고, 그 정중앙 값을 찍는다.
- (2) 판단: "정답보다 큰가, 작을까?"를 보고 범위를 절반으로 좁힌다.
- (3) 결과: 이 과정을 반복하면 정답에 아주 가까워진다.

## Lab: 주사위 합이 6이 되는 경우

- 주사위 2개를 던졌을 때, 합이 6이 되는 경우를 전부 출력하여 보자.

```
1. for a in range(1, 7):
2.     for b in range(1, 7):
3.         if a + b == 6:
4.             print("첫 번째 주사위-(a) 두 번째 주사위-(b) ")
```

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "C:\Users\USER\Download\import\_random.py"

첫 번째 주사위=1 두 번째 주사위=5  
첫 번째 주사위=2 두 번째 주사위=4  
첫 번째 주사위=3 두 번째 주사위=3  
첫 번째 주사위=4 두 번째 주사위=2  
첫 번째 주사위=5 두 번째 주사위=1

## Lab: 모든 조합 출력하기

```
persons = ["Kim", "Park", "Lee", "Choi"]  
restaurants = ["Korean", "American", "French", "Chinese"]
```

```
1. persons = ["Kim", "Park", "Lee"]  
2. restaurants = ["Korean", "American", "French"]  
3. for person in persons:  
4.     for restaurant in restaurants:  
5.         print(person + "이 " + restaurant + " 식당을 방문")
```

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "C:\Users\USER\Download\import\_random.py"

Kim이 Korean 식당을 방문  
Kim이 American 식당을 방문  
Kim이 French 식당을 방문  
Park이 Korean 식당을 방문  
Park이 American 식당을 방문  
Park이 French 식당을 방문  
Lee이 Korean 식당을 방문  
Lee이 American 식당을 방문  
Lee이 French 식당을 방문

## Lab: 소수 찾기

- 2부터 시작하여 50개의 소수를 찾는 프로그램을 작성해보자.

```
1. N_PRIMES = 50  
2. number = 2  
3. count = 0  
4. while count < N_PRIMES:  
5.     divisor = 2  
6.     prime = True  
7.     while divisor < number:  
8.         if number % divisor == 0:  
9.             prime = False  
10.            break  
11.         divisor += 1  
12.     if prime:  
13.         count += 1  
14.         print(number, end=" ")  
15.         number += 1
```

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "C:\Users\USER\Download\import\_random.py"

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199 211 223 227 229

## Lab: 파이 계산하기

- 파이를 계산하는 것은 무척 시간이 많이 걸리는 작업으로 주로 슈퍼 컴퓨터의 성능을 시험하는 용도로 사용된다. 지금은 컴퓨터의 도움으로 10조개의 자릿수까지 계산할 수 있다. 파이를 계산하는 가장 고전적인 방법은 Gregory-Leibniz 무한 수열을 이용하는 것이다.

```
1. divisor = 1.0  
2. dividend = 4.0  
3. sum = 0.0  
4. loop_count = int(input("반복횟수:"))  
5. while loop_count > 0:  
6.     sum = sum + dividend / divisor  
7.     dividend = -1.0 * dividend  
8.     divisor = divisor * 2  
9.     loop_count = loop_count - 1  
10. print("pi = %.1f" % sum)
```

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "C:\Users\USER\Download\import\_random.py"

반복횟수:10000  
pi = -3.14159

## Lab: 도박상의 확률

- 어떤 사람이 50달러를 가지고 라스베가스에서 게임을 한다고 하자. 한 번의 게임에 1달러를 건다고 가정하자. 돈을 딸 확률은 0.5이라고 가정하자. 한번 라스베가스에 가면, 가진 돈을 다 잃거나 목표 금액인 250달러에 도달할 때까지 게임을 계속한다. 어떤 사람이 라스베가스에 100번을 갔다면 몇 번이나 250달러를 따서 돌아올 수 있을까?

```
1. import random  
2. initial_money = 50  
3. goal = 250  
4. wins = 0  
5. for i in range(100):  
6.     cash = initial_money  
7.     while cash > 0 and cash < goal:  
8.         number = random.randint(1, 2)  
9.         if number == 1:  
10.            cash = cash + 1  
11.         else:  
12.            cash = cash - 1  
13.         if cash == goal: wins = wins + 1  
14.     print("초기 금액 $50" % initial_money)  
15.     print("최종 금액 $%s" % goal)  
16.     print("100번 중에서 %d번 성공" % wins)
```

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "C:\Users\USER\Download\import\_random.py"

초기 금액 \$50  
최종 금액 \$250  
100번 중에서 11번 성공

## 이번 장에서 배운 것

- 문장들을 반복 실행하려면 for 문이나 while 문을 사용한다.
- 반복 실행되는 문장들을 들여쓰기 하여야 한다.
- for 문은 반복 회수를 정해져있을 때 유용하다.
- while 문은 반복 조건이 정해져 있을 때 유용하다.
- 반복문의 초입에서 조건식은 검사된다.



# 5주차: 함수

readme.md

## 5. 함수

함수는 자주 반복되는 특정 작업 명령어들을 하나로 묶어 이름을 붙여놓은 '코드 재사용 상자'입니다. def 키워드로 정의하고 필요할 때 이름을 불러 호출합니다. 함수는 외부에서 데이터를 입력받는 '매개변수'를 가질 수 있으며, 작업이 끝난 후 계산된 결과값을 return을 통해 돌려줄 수도 있습니다.

예시

```
def add_numbers(a, b):  
    return a + b  
  
result = add_numbers(5, 3)  
print("합계:", result)
```

## Commits

History for Python / 0410 on main

All users All time

Commits on Jun 10, 2026

### Update readme.md

sgdn12 authored 2 hours ago

Verified a9438b6

Commits on Apr 15, 2026

### Add files via upload

sgdn12 authored on Apr 15

Verified 7fe03ea

### Create readme.md

sgdn12 authored on Apr 15

Verified fa1b8e8

중간점검

1. 함수란 무엇인가?

- 프로그래밍에서 특정한 작업을 수행하기 위해 독립적으로 설계된 코드의 집합

2. 함수를 사용하는 이유는 무엇인가?

- 재사용성이 높으며 모듈화가 잘 되어있어서 코드 구조가 한눈에 보이고 오류난 해당 함수 한 곳만 고치면 프로그램 전체에 반영되기 때문이다.

3. 함수에 전달되는 값을 무엇이라고 하는가?

- 인수(Argument) : 함수를 정의할 때 사용하는 변수 이름

- 매개변수(Parameter) : 함수를 호출할 때 실제로 전달하는 구체적인 값

4. 함수 안에서 전달되는 값을 받는 변수를 무엇이라고 하는가?

- 매개변수(Argument)

5. 사용자로부터 2개의 정수를 받아서 반환하는 함수를 작성해보자.

```
1 def get_two_numbers():  
2     num1 = int(input("첫 번째 정수를 입력하세요: "))  
3     num2 = int(input("두 번째 정수를 입력하세요: "))  
4     return num1, num2  
5  
6 result1, result2 = get_two_numbers()  
7 print(f"입력받은 값은 (result1)와 (result2)입니다.")
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
oads/import random.py  
사각형의 면적: 50  
PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "c:/Us  
oads/import random.py  
첫 번째 정수를 입력하세요: 12  
두 번째 정수를 입력하세요: 22  
입력받은 값은 12와 (과) 22입니다.  
PS C:\Users\USER>
```

6. 주어진 사각형의 면적을 계산하는 함수 get\_rect\_area(w, h)를 정의해보자.

여기서 w는 너비, h는 높이이다.

```
1 def get_rect_area(w, h):  
2     """  
3     사각형의 너비와 높이를 곱하여 면적을 계산합니다.  
4     """  
5     return w * h  
6  
7 area = get_rect_area(5, 10)  
8 print(f"사각형의 면적: {area}") # 출력: 50
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
oads/import random.py  
사각형의 면적: 50  
PS C:\Users\USER>
```

7. main() 함수를 정의하고 main() 함수 안에서 get\_rect\_area()를 호출해보자

```
1 def get_rect_area(w, h):  
2     return w * h  
3  
4 def main():  
5     print("--- 사각형 면적 계산기 ---")  
6     width = int(input("너비를 입력하세요: "))  
7     height = int(input("높이를 입력하세요: "))  
8  
9     area = get_rect_area(width, height)  
10    print(f"너비 {width}, 높이 {height}인 사각형의 면적은 {area}입니다.")  
11  
12 if __name__ == "__main__":  
13    main()  
14
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
--- 사각형 면적 계산기 ---  
너비를 입력하세요: 10  
높이를 입력하세요: 10  
너비 10, 높이 10인 사각형의 면적은 100입니다.  
PS C:\Users\USER>
```

8. 인수와 매개 변수는 다시 한번 설명해보자

-인수는 정의할 때 사용하는 변수이름

-매개변수는 함수를 호출할 때 실제로 전달하는 구체적인 값

9. 디폴트 인수란 무엇인가? 예를 들어보자.

-디폴트 인수 : 함수를 정의할 때 매개변수에 기본값을 미리 할당해주는 것



```

1 def get_rect_area(w, h=10):
2     return w * h
3
4 print(get_rect_area(5, 5))
5
6 print(get_rect_area(5))

```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "c:\Users\USER\Downloads\import\_random.py"

25.0

10. 키워드 인수란 무엇인가? 예를 들어보자.

-키워드 인수란 함수를 호출할 때 인수의 값을 위치로 구별하는 것이 아니라, 매개변수의 이름을 직접 지정하여 전달하는 방식

```

1 def introduce(name, age, city="서울"):
2     print(f"이름: {name}, 나이: {age}, 거주지: {city}")
3
4 introduce("철수", 25)
5
6 introduce(age=30, name="영희")
7
8 introduce(name="지민", age=22, city="부산")

```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

PS C:\Users\USER> oads/import\_random.py

이름: 철수, 나이: 25, 거주지: 서울  
이름: 영희, 나이: 30, 거주지: 서울  
이름: 지민, 나이: 22, 거주지: 부산

11. 매개 변수 앞에 \* 기호가 있다면 무슨 의미인가?

-가변 인수라고 부른다. 가변 인수는 함수를 호출할 때 인수를 몇 개나 넣을지 미리 알 수 없는 경우가 있다 이때 \*를 붙이면, 입력되는 여러 개의 인수를 하나의 튜플로 묶어서 받게 된다.

12. 값을 반환하는 키워드는 무엇인가?

-return

13. x와 y를 받아서 x+y, x-y 값을 반환하는 함수 addsub(x, y)를 정의해보자.

```

1 def addsub(x, y):
2     return x + y, x - y
3
4 result_add, result_sub = addsub(10, 5)
5 print(result_add, result_sub)

```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "c:\Users\USER\Downloads\import\_random.py"

15.0 5.0

14. 지역 변수와 전역 변수는 어떻게 다른가?

-지역 변수는 함수 내부에서 정의된 변수이다.

범위: 해당 함수 안에서만 사용할 수 있습니다.

수명: 함수가 호출될 때 만들어지고, 함수 실행이 끝나면 메모리에서 사라집니다.

특징: 함수 바깥에서는 이 변수가 있는지조차 알 수 없습니다.

-전역 변수는 함수 바깥에서 정의된 변수이다.

범위: 프로그램 전체(모든 함수)에서 읽고 사용할 수 있습니다.

수명: 프로그램이 시작될 때 만들어지고, 프로그램이 종료될 때까지 유지됩니다.

특징: 어디서든 접근 가능하지만, 함수 안에서 값을 수정하려면 별도의 처리가 필요합니다.

15. 함수 안에서 전역 변수의 값을 읽을 수 있는가?

-언제든 가능하다. 파일의 규칙상 함수 내부에서 특정 변수를 찾을 때, 자기 안에 해당 변수가 없으면 자동으로 함수 바깥에서 그 변수를 찾기 때문이다.

16. 함수 안에서 전역 변수의 값을 변경하면 어떻게 되는가?

-함수 안에서 전역 변수와 똑같은 이름의 변수에 값을 넣으면, 파일은 전역 변수를 수정하는 대신 이름이 같은 새로운 지역 변수를 만들어 버린다.

## Lab: 피자 크기 비교

```

1 def main():
2     print("20cm 피자 2개의 면적:", get_area(20)+get_area(20))
3     print("30cm 피자 1개의 면적:", get_area(30))
4
5 def get_area(radius):
6     if radius > 0:
7         area = 3.14*radius**2
8     else:
9         area = 0
10    return area
11
12 main()

```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "c:\Users\USER\Downloads\import\_random.py"

20cm 피자 2개의 면적: 2512.0  
30cm 피자 1개의 면적: 2826.0

## Lab: 환영 문자열 출력 함수

```

1 def display(msg, count=1):
2     for k in range(count):
3         print(msg)
4     display("환영합니다.", 5)

```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "c:\Users\USER\Downloads\import\_random.py"

환영합니다.  
환영합니다.  
환영합니다.  
환영합니다.  
환영합니다.

## Lab: 이분법

```

1 def f(x):
2     return x**2-x-1
3
4 def bisection_method(a, b, error):
5     if f(a)*f(b) > 0:
6         print("구간에서 근을 찾을 수 없습니다.")
7     else:
8         while (b - a)/2.0 > error:
9             midpoint = (a + b)/2.0
10            if f(midpoint) == 0:
11                return(midpoint)
12            elif f(a)*f(midpoint) < 0:
13                b = midpoint
14            else:
15                a = midpoint
16            return(midpoint)
17
18 answer = bisection_method(1, 2, 0.0001)
19 print("x**2-x-1의 근:", answer)

```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

PS C:\Users\USER> & C:\Users\USER\AppData\Local\Programs\Python\Python314\python.exe "c:\Users\USER\Downloads\import\_random.py"

1.618033988749895

## Lab: 구조화 프로그래밍 실습

C:\Users\USER> OneDrive\문서> Untitled-1.py > main

```
1 def menu():
2     print("3. 종료")
3     selection = int(input("메뉴를 선택하세요: "))
4     return selection
5
6 def ctof(c):
7     temp = c*9.0/5.0 + 32
8     return temp
9
10 def ftoc(f):
11     temp = (f-32.0)*5.0/9.0
12     return temp
13
14 def input_f():
15     f = float(input("화씨 온도를 입력하세요: "))
16     return f
17
18 def input_c():
19     c = float(input("섭씨 온도를 입력하세요: "))
20     return c
21
22 def main():
23     while True:
24         index = menu()
25         if index == 1:
26             t = input_c()
27             t2 = ctof(t)
28             print("화씨 온도 = ", t2, "\n")
29         elif index == 2:
30             t = input_f()
31             t2 = ftoc(t)
32             print("섭씨 온도 = ", t2, "\n")
33         else:
34             break
35     main()
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
1. 섭씨 온도->화씨 온도
2. 화씨 온도->섭씨 온도
3. 종료
메뉴를 선택하세요: 2
화씨 온도를 입력하세요: 100.0
섭씨 온도 = 37.77777777777778
```

```
1. 섭씨 온도->화씨 온도
2. 화씨 온도->섭씨 온도
3. 종료
메뉴를 선택하세요: 3
```

## Lab: 주급 계산 프로그램

```
1 def weeklyPay( rate, hour ):
2     money = 0
3     if (hour > 30):
4         money = rate*30 + 1.5*rate*(hour-30)
5     else:
6         money = rate*hour
7     return money
8
9 rate = int(input("시급을 입력하세요: "))
10 hour = int(input("근무 시간을 입력하세요: "))
11 print("주급은 " + str(weeklyPay(rate, hour)))
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
시급을 입력하세요: 11050
근무 시간을 입력하세요: 12
주급은 132600
```

## Lab: 여러 개의 값 반환

```
1 def get_info():
2     name = input("이름:")
3     age = int(input("나이:"))
4     return name, age
5
6 st_name, st_age = get_info()
7 print("이름은 ", st_name, "이고 나이는 ", st_age, "살입니다.")
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
rams/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
이름:김동교
나이:20
이름은 김동교 이고 나이는 20 살입니다.
```

# 6주차: 파이썬 자료구조(리스트)

## 6. 파이썬 자료구조 I: 리스트

리스트는 여러 개의 데이터를 대괄호([]) 안에 순서대로 나열하여 저장하는 자료구조입니다. 각 데이터의 위치를 나타내는 번호인 '인덱스'는 0부터 시작하며, 이를 통해 특정 위치의 값을 읽거나 수정할 수 있습니다. append()로 새 값을 추가하거나 sort()로 정렬할 수 있으며, 리스트 안에 또 다른 리스트를 넣어 행렬 형태의 2차원 리스트를 만들 수도 있습니다.

예시

```
fruits = ["사과", "바나나"]
fruits.append("포도")
print(fruits[0])
```

## Commits

History for Python / 0417 on main

All users All time

Commits on Jun 10, 2026

Update readme.md

sgdn12 authored 2 hours ago

Verified dbb1cf0

Commits on Apr 23, 2026

Add files via upload

sgdn12 authored on Apr 23

Verified fd9b43c

Create readme.md

sgdn12 authored on Apr 23

Verified b3c2921

## Lab: 콘테스트 평가

- 학생들의 점수가 리스트에 저장되어 있다고 가정하고 최소값과 최대값을 리스트에서 제거하는 프로그램을 작성해보자.

제거전 [10.0, 9.0, 8.3, 7.1, 3.0, 9.0]  
제거후 [9.0, 8.3, 7.1, 9.0]

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 scores = [10.0, 9.0, 8.3, 7.1, 3.0, 9.0]
2 print("제거전", scores)
3 scores.remove(max(scores))
4 scores.remove(min(scores))
5 print("제거후", scores)
```

## Lab: 리스트로 스택 흉내내기

- 스택은 데이터 구조 중의 하나로서 데이터를 추가한 순서와 삭제하는 순서가 일치하는 반대인 데이터 구조이다. 파이썬에는 내장 스택 유형이 없지만, 리스트를 리스트로 구현할 수 있다.

```
1 stack = []
2 for i in range(3):
3     fruit = input("과일을 입력하십시오: ")
4     stack.append(fruit)
5 for i in range(3):
6     print(stack.pop())
```

## Lab: 리스트에서 2번째로 큰 수 찾기

- 점수들이 저장된 리스트에서 두 번째로 큰 수를 찾아보자.

두 번째로 큰 수= 15

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 list1 = [1, 2, 3, 4, 15, 99]
2 list1.sort()
3 print("두번째로 큰 수=", list1[-2])
```

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 list1 = [1, 2, 3, 4, 15, 99]
2 list1.remove(max(list1))
3 print("두번째로 큰 수=", max(list1))
```

## 중간점검

- 함수를 무엇인가?
  - 특정 작업을 위해 재사용 가능한 명령어 묶음을 말한다.
- 함수를 사용하는 이유는 무엇인가?
  - 코드의 재사용성, 구조화, 유지보수성을 위해서 사용한다.

## Lab: 성적 처리 프로그램

- 학생들의 성적을 사용자로부터 입력받아서 리스트에 저장한다. 성적의 평균을 구하고 최대점수, 최소점수, 80점 이상 성적 받은 학생의 숫자를 계산하여 출력해보자.

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 STUDENTS = 5
2 list = []
3 count = 0
4 for i in range(STUDENTS):
5     value = int(input("성적을 입력하십시오: "))
6     list.append(value)
7
8 print("\n성적 평균=", sum(list) / len(list))
9 print("최대점수=", max(list))
10 print("최소점수=", min(list))
11
12 for score in list:
13     if score >= 80:
14         count += 1
15 print("80점 이상=", count)
```



## Lab: 친구 관리 프로그램

- 파이썬을 이용하여 친구들을 관리하는 프로그램을 작성하여 보자. 친구 관리 프로그램은 다음과 같은 메뉴를 가져야 한다.

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
6 print("2. 친구 추가")
7 print("3. 친구 삭제")
8 print("4. 이름변경")
9 print("9. 종료")
10 menu = int(input("메뉴를 입력하시오: "))
11 if menu == 1:
12     print(friends)
13 elif menu == 2:
14     name = input("이름을 입력하시오: ")
15     friends.append(name)
16 elif menu == 3:
17     del_name = input("삭제하고 싶은 이름을 입력하시오: ")
18     if del_name in friends:
19         friends.remove(del_name)
20 else:
21     print("이름이 발견되지 않음.")
22 elif menu == 4:
23     old_name = input("변경하고 싶은 이름을 입력하시오: ")
24     if old_name in friends:
25         index = friends.index(old_name)
26         new_name = input("새로운 이름을 입력하시오: ")
27         friends[index] = new_name
28     else:
29         print("이름이 발견되지 않음")

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python Deb...

9. 종료
메뉴를 입력하시오: 1
[]
-----
1. 친구 리스트 출력
2. 친구 추가
3. 친구 삭제
4. 이름변경
9. 종료
메뉴를 입력하시오: 2
이름을 입력하시오: 김동고
-----
1. 친구 리스트 출력
2. 친구 추가
```

## Lab: 피타고라스 삼각형

- 피타고라스의 정리를 만족하는 삼각형들을 모두 찾아보자. 삼각형 한 변의 길이는 1 부터 30 이하이다.

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 result = [(x,y,z) for x in range(1, 30) for y in range(x,30) for z
2 print(result)

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python

PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
[(3, 4, 5), (5, 12, 13), (6, 8, 10), (7, 24, 25), (8, 15, 17), (9, 12, 15), (10, 24, 26), (12, 16, 20), (15, 20, 25), (20, 21, 29)]
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>

C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 new_list = []
2 for x in range(1, 30):
3     for y in range(x, 30):
4         for z in range(y, 30):
5             if x**2+y**2==z**2:
6                 new_list.append((x, y, z))
7 print(new_list)

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python

PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
[(3, 4, 5), (5, 12, 13), (6, 8, 10), (7, 24, 25), (8, 15, 17), (9, 12, 15), (10, 24, 26), (12, 16, 20), (15, 20, 25), (20, 21, 29)]
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```

## Lab: 리스트 함축 사용하기

- 0부터 99까지의 정수 중에서 2의 배수이고 동시에 3의 배수인 수들을 모아서 리스트로 만들어보자.

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 numbers = [x for x in range(100) if x % 2 == 0 and x % 3 == 0]
2 print(numbers)

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python
Python

PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
[0, 6, 12, 18, 24, 30, 36, 42, 48, 54, 60, 66, 72, 78, 84, 90, 96]
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```

## Lab: 누적값 리스트 만들기

- i번째 요소가 원래 리스트의 0부터 i번째 요소까지의 합계인 리스트를 생성하는 프로그램을 작성하라.

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 list1 = [10, 20, 30, 40, 50]
2 list2 = [sum(list1[0:x+1]) for x in range(0, len(list1))]
3 print("원래 리스트:", list1)
4 print("새로운 리스트:", list2)

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python
Python

rive/문서/Untitled-1.py
원래 리스트: [10, 20, 30, 40, 50]
새로운 리스트: [10, 30, 60, 100, 150]
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```

## Lab: 리스트 슬라이싱

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2 reversed = numbers[::-2]
3 print(reversed)
4

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python
Python

/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
[10, 8, 6, 4, 2]

C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2 numbers[1:] = []
3 print(numbers)
4

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python
Python

PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
[1]
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```

## Lab: 리스트 변경 함수

```
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > modify
1 salaries = [200, 250, 300, 280, 500]
2 def modify(values, factor):
3     for i in range(len(values)):
4         values[i] = values[i]*factor
5     print("인상전", salaries)
6     modify(salaries, 1.3)
7     print("인상후", salaries)

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
Python
Python
Python

PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneDrive/문서/Untitled-1.py
인상전 [200, 250, 300, 280, 500]
인상후 [260.0, 325.0, 390.0, 364.0, 650.0]
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```





## Lab: Tic-Tac-Toe

- 다음과 같이 텍스트 모드에서 컴퓨터와 사람이 Tic-Tac-Toe 게임을 할 수 있는 프로그램을 작성하여 보자.

```
Untitled-1.py X
C:\Users\USER> OneDrive > 문서 > Untitled-1.py > ...
1 board = [[' ' for x in range(3)] for y in range(3)]
2
3 while True:
4     for r in range(3):
5         print(" " + board[r][0] + " | " + board[r][1] + " | " + board[r][2])
6         if r != 2:
7             print("----|---|---")
8     try:
9         x = int(input("다음 수의 x좌표를 입력하십시오(0~2): "))
10        y = int(input("다음 수의 y좌표를 입력하십시오(0~2): "))
11    except ValueError:
12        print("숫자를 입력해주세요!")
13        continue
14    if x < 0 or x > 2 or y < 0 or y > 2:
15        print("0, 1, 2 중에서 입력하세요.")
16        continue
17    if board[x][y] != ' ':
18        print("이미 차 있는 위치입니다. 다시 선택하세요.")
19        continue
20    else:
21        board[x][y] = 'X'
22    done = False
23    for i in range(3):
24        for j in range(3):
25            if board[i][j] == ' ' and not done:
26                board[i][j] = 'O'
27                done = True
28                break
29    if done: break
30    full = True
31    for row in board:
32        if ' ' in row:
33            full = False
34    if full:
35        print("게임 종료!")
36        break
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
>
다음 수의 x좌표를 입력하십시오(0~2): 1
다음 수의 y좌표를 입력하십시오(0~2): 0
O | O | O
----|---|---
X | X | X
----|---|---
다음 수의 x좌표를 입력하십시오(0~2):
```



## Lab: 전치 행렬 계산

- 행렬의 전치 연산을 파이썬으로 구현해보자. 인공 지능을 하려면 행렬에 대하여 잘 알아야 한다. 중첩된 for 루프를 이용하면 행렬의 전치를 계산할 수 있다.

```
1 transposed = []
2 matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
3 print("원래 행렬=", matrix)
4 for i in range(len(matrix[0])):
5     transposed_row = []
6     for row in matrix:
7         transposed_row.append(row[i])
8     transposed.append(transposed_row)
9 print("전치 행렬=", transposed)
```

PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE

TERMINAL

```
>
원래 행렬= [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
전치 행렬= [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```

## 7주차: 파이썬 자료구조(튜플, 딕셔너리, 세트, 문자열)

## 7. 자료구조 II (튜플, 세트, 딕셔너리)

리스트 외에도 다양한 용도의 자료구조가 존재합니다. 튜플(`()`)은 리스트와 같지만 한 번 저장하면 수정할 수 없어 안전한 데이터 보관에 쓰입니다. 세트(`()`)는 중복을 자동으로 제거하며 순서가 없는 집합 자료구조입니다. 딕셔너리(`{key: value}`)는 단어 사전처럼 '키'와 '값'을 쌍으로 묶어 저장하여 고유한 키를 통해 원하는 값을 빠르게 찾아낼 때 사용합니다.

## 예시

```
phone_book = {"홍길동": "010-1234-5678", "이순신": "010-9876-5432"}
print(phone_book["홍길동"])
```

 Lab: 문자열의 공통 문자

```
1 s1=input("첫 번째 문자열:")
2 s2=input("두 번째 문자열:")
3 list1 = list( set(s1)&set(s2))
4 print("\n공통적인 글자:",end=" ")
5
6 for i in list1:
7     print(i, end=" ")
```

 Lab: 분사열의 공동 분사

[illegible]

 Lab: 영화 사전

```
1 english_dict = { }
2 english_dict["one"] = "하나"
3 english_dict["two"] = "둘"
4 english_dict["three"] = "셋"
5
6 word = input("단어를 입력하십시오 : ")
7 print(english_dict[word])
8
```

Lab: 학생 성적 처리

```

1  def main():
2      address_book = {}
3      while True:
4          user = display_menu()
5          if user == 1:
6              name, number = get_contact()
7              address_book[name] = number
8          elif user == 2:
9              name, number = get_contact()
10             address_book.pop(name)
11          elif user == 3:
12              pass
13          elif user == 4:
14              for key in sorted(address_book):
15                  print(key, "의 전화번호 :", address_book[key])
16          else:
17              break
18  def get_contact():
19      name = input("이름 : ")
20      number = input("전화번호 : ")
21      return name, number
22  def display_menu():
23      print("1. 연락처 추가")
24      print("2. 연락처 삭제")
25      print("3. 연락처 목록")
26      print("4. 연락처 종료")
27      print("5. 종료")
28      select = int(input("메뉴 항목을 선택하십시오 : "))
29      return select

```

## Commits

History for Python / 0508 on main

**Update readme.md**

 sgdn12 authored 2 hours ago

**Add files via upload**  
sgdn12 authored last month

Create [readme.md](#)

End of commit history for this file



## Lab: 단어 카운터 만들기

```
1 text_data = "Create the highest, grandest vision possible for your 1"
2 word_dic = {}
3 for w in text_data.split():
4     if w in word_dic:
5         word_dic[w] += 1
6     else:
7         word_dic[w] = 1
8 for w, count in sorted(word_dic.items()):
9     print(w, "의 등장횟수=", count)
10 from collections import Counter
11 text_data = "Create the highest, grandest vision possible for your 1"
12 a = Counter(text_data.split())
13 print(a)
```

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
▼ TERMINAL
e': 1, 'become': 1, 'what': 1, 'believe': 1})
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER
/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneD
rive/문서/Untitled-1.py
Create 의 등장횟수= 1
because 의 등장횟수= 1
beco 의 등장횟수= 1
believe 의 등장횟수= 1
for 의 등장횟수= 1
grandest 의 등장횟수= 1
highest, 의 등장횟수= 1
life, 의 등장횟수= 1
me 의 등장횟수= 1
possible 의 등장횟수= 1
the 의 등장횟수= 1
vision 의 등장횟수= 1
what 의 등장횟수= 1
you 의 등장횟수= 2
your 의 등장횟수= 1
Counter({'you': 2, 'Create': 1, 'the': 1, 'highest': 1, 'grandest': 1,
'vision': 1, 'possible': 1, 'for': 1, 'your': 1, 'life': 1, 'because': 1, 'become': 1, 'what': 1, 'believe': 1})
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```

## Lab: 트위터 메시지 처리

- 일반적으로 부정적인 감정보다 긍정적인 것보다 적은 양의 단어를 포함한다고 한다. 트윗에서 단어의 개수를 추출하여서 발신자의 감정을 판단해보자.

```
t = "Python is very easy and powerful"
length = len(t.split(" "))
print(length)
```

```
1 length = len(t.split(" "))
2
3
4
5 print(length)
```

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
▼ TERMINAL
{'alphas': 29, 'digits': 0, 'spaces': 6}
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER
/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneD
rive/문서/Untitled-1.py
6
```

## Lab: OTP 발생 프로그램

- 일회용 암호 (OTP) 프로그램을 작성해보자.

3482

```
1 import random
2 s = "0123456789"
3 passlen = 4
4
5 p = "".join(random.sample(s, passlen))
6 print(p)
```

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
▼ TERMINAL
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER
/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneD
rive/문서/Untitled-1.py
5648
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src>
```

## Lab: 이메일 주소 분석

- 이메일 주소에서 아이디와 도메인을 구분하는 프로그램을 작성하여 보자.

이메일 주소를 입력하십시오: aaa@google.com  
아이디: aaa  
도메인: google.com

```
1 address = input("이메일 주소를 입력하십시오: ")
2 (id, domain) = address.split("@")
3 print(address)
4 print("아이디:", id)
5 print("도메인:", domain)
```

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
▼ TERMINAL
/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneD
rive/문서/Untitled-1.py
이메일 주소를 입력하십시오: gdg8831@gmail.com
아이디: gdg8831
도메인: gmail.com
```

## Lab: 문자열 분석

- 문자열 안에 있는 문자의 개수, 숫자의 개수, 공백의 개수를 계산하는 프로그램을 작성하여 보자.

문자열을 입력하십시오: A picture is worth a thousand words.  
("digits": 0, "spaces": 6, "alphas": 29)

```
1 sentence = input("문자열을 입력하십시오: ")
2 table = {"alphas": 0, "digits": 0, "spaces": 0}
3 for i in sentence:
4     if i.isalpha():
5         table["alphas"] += 1
6     if i.isdigit():
7         table["digits"] += 1
8     if i.isspace():
9         table["spaces"] += 1
10 print(table)
```

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
▼ TERMINAL
PS C:\Users\USER\OneDrive\Desktop\2026964001\0410\src> & C:/Users/USER
/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneD
rive/문서/Untitled-1.py
문자열을 입력하십시오: A picture is worth a thousand words.
{'alphas': 29, 'digits': 0, 'spaces': 6}
```

## Lab: 회문 검사하기

- 회문(palindrome)은 앞으로 읽으나 뒤로 읽으나 동일한 문장이다. 예를 들어서 "mom", "civic", "dad" 등이 회문의 예이다. 사용자가로부터 문자열을 입력받고 회문인지 검사하는 프로그램을 작성하여 보자.

문자열을 입력하십시오: dad  
회문입니다.

```
1 s = input("문자열을 입력하십시오: ")
2 si = s[::-1]
3 if s == si:
4     print("회문입니다.")
5 else:
6     print("회문이 아닙니다.")
```

## Lab: 머리 글자어 만들기

- 머리 글자어(acronym)은 NATO(North Atlantic Treaty Organization)처럼 각 단어의 첫글자를 모아서 만든 문자열이다. 사용자가 문장을 입력하면 해당되는 머리 글자어를 출력하는 프로그램을 작성하여 보자.

문자열을 입력하십시오: North Atlantic Treaty Organization  
NATO

```
1 phrase = input("문자열을 입력하십시오: ")
2 acronym = ""
3
4 for word in phrase.upper().split():
5     acronym += word[0]
6 print(acronym)
```

```
PROBLEMS TERMINAL OUTPUT DEBUG CONSOLE
▼ TERMINAL
/AppData/Local/Programs/Python/Python314/python.exe c:/Users/USER/OneD
rive/문서/Untitled-1.py
문자열을 입력하십시오: North Atlantic Treaty Organization
```



# 8주차: 객체와 클래스

## 8. 객체와 클래스

객체지향 프로그래밍은 데이터(속성)와 기능(메서드)을 하나의 단위로 묶어 관리하는 방식입니다. 클래스는 이러한 객체를 찍어내기 위한 '설계도'이며, 설계도를 통해 실제로 메모리에 만들어진 객체를 인스턴스라고 합니다. 클래스 내부의 `init()` 함수는 객체가 처음 태어날 때 초기 속성을 설정해 주는 생성자 역할을 합니다.

예시

```
class Dog:
    def __init__(self, name):
        self.name = name

my_dog = Dog("맥스")
print(my_dog.name)
```

### Lab:TV 클래스 정의

```
1 class Television:
2     def __init__(self, channel, volume, on):
3         self.channel = channel
4         self.volume = volume
5         self.on = on
6     def show(self):
7         print(self.channel, self.volume, self.on)
8     def setchannel(self, channel):
9         self.channel = channel
10    def getchannel(self):
11        return self.channel
12    t = Television(0, 10, True)
13    t.show()
14    t.setchannel(11)
15    t.show()
```

### Lab: 원 클래스 정의

```
1 import math
2 class Circle:
3     def __init__(self, radius = 0):
4         self.radius = radius
5     def getArea(self):
6         return math.pi * self.radius * self.radius
7     def getPerimeter(self):
8         return 2 * math.pi * self.radius
9    c = Circle(10)
10    print("원의 면적", c.getArea())
11    print("원의 둘레", c.getPerimeter())
```

### Lab:TV 클래스 정의

```
1 class Car:
2     def __init__(self, speed, color, model):
3         self.speed = speed
4         self.color = color
5         self.model = model
6     def drive(self):
7         self.speed = 60
8 myCar = Car(0, "blue", "E-class")
9 print("자동차 객체를 생성하였습니다.")
10 print("자동차의 속도는", myCar.speed)
11 print("자동차의 색상", myCar.color)
12 print("자동차의 모델", myCar.model)
13 myCar.drive()
14 print("자동차의 속도는", myCar.speed)
```

### Lab:은행 계좌

```
1 class BankAccount:
2     def __init__(self):
3         self._balance = 0
4     def withdraw(self, amount):
5         self._balance -= amount
6         print("통장에 ", amount, "가 입금되었음")
7         return self._balance
8     def deposit(self, amount):
9         self._balance += amount
10        print("통장에서 ", amount, "가 출금되었음")
11        return self._balance
12    a = BankAccount()
13    a.deposit(100)
14    a.withdraw(10)
```

| Commits                                             |                  |
|-----------------------------------------------------|------------------|
| History for Python / 0515 on main                   |                  |
| Commits on Jun 10, 2026                             |                  |
| Update readme.md<br>sgdn12 authored 2 hours ago     | Verified b5c8b31 |
| Commits on May 22, 2026                             |                  |
| Add files via upload<br>sgdn12 authored 3 weeks ago | Verified 7f77dda |
| Commits on May 15, 2026                             |                  |
| Create readme.md<br>sgdn12 authored last month      | Verified 42521f7 |

## Lab: 클래스 변수

```
class Dog:
    king = "Bulldog"
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

## Lab:은행 계좌

```
1 class Vector2D:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5     def _add_(self, other):
6         return Vector2D(self.x + other.x, self.y + other.y)
7     def _sub_(self, other):
8         return Vector2D(self.x - other.x, self.y - other.y)
9     def _eq_(self, other):
10        return self.x == other.x and self.y == other.y
11    def _str_(self):
12        return '%g, %g' % (self.x, self.y)
13    u = Vector2D(0,1)
14    v = Vector2D(1,0)
15    w = Vector2D(1,1)
16    a = u + v
17    print(a)
```

# 프로젝트 : 주사위 게임

# 주사위 게임 코드 설명

## 1~4행

- 1행: random — 주사위 눈(1~6)을 무작위로 뽑기 위해 사용합니다.
- 2행: time — AI가 주사위를 굴릴 때 멈춰있는 효과(time.sleep)를 주기 위해 사용합니다.
- 3행: tkinter as tk — 파이썬 기본 GUI 라이브러리를 tk라는 축약어로 가져옵니다.
- 4행: messagebox — (현재 코드에선 쓰이지 않지만) 알림창을 띄울 때 쓰는 도구입니다.

```
import random
import time
import tkinter as tk
from tkinter import messagebox
```

## 7~30행

- 6~7행: DiceGameGUI 클래스를 만들고, 창이 생성될 때 실행될 초기화 함수(\_\_init\_\_)를 정의합니다.
- 8~11행: 게임 창의 제목, 크기(너비 450, 높이 500), 배경색(연한 푸른빛 공대 느낌의 회색)을 지정합니다.
- 14~20행: 화면 맨 위에 들어갈 대형 제목 글자(🎲 주사위 대결 게임 🎲)의 폰트, 배경색, 글자색을 설정합니다.
- 21행: pack(pady=20) — 제목 레이블을 화면에 배치하고, 위아래 여백을 20만큼 줍니다.

```
class DiceGameGUI:

    def __init__(self, window):
        self.window = window
        self.window.title("🎲 AI vs Player 주사위 대결")
        self.window.geometry("450x500") # 주사위가 줄어들어 세로 길이를 조금 줄였습니다.
        self.window.configure(bg="#f0f4f8")

        # 타이틀 레이블
        self.title_label = tk.Label(
            window,
            text="🎲 주사위 대결 게임 🎲",
            font=("Helvetica", 20, "bold"),
            bg="#f0f4f8",
            fg="#333333",
        )
        self.title_label.pack(pady=20)
```



# 주사위 게임 코드 설명

## 23~25행

- 24행: 플레이어 영역과 AI 영역을 가로로 나란히 묶어줄 투명한 바구니(Frame)를 만듭니다.
- 25행: 이 바구니를 화면 가운데에 배치합니다.

```
# 게임 판 (프레임)
self.board_frame = tk.Frame(window, bg="#f0f4f8")
self.board_frame.pack(pady=10)
```

## 27~50행

- 28~38행: 테두리가 있는 상자 형태의 LabelFrame을 만들어 플레이어 구역을 정의합니다. 글자는 파란색(fg="#2b6cb0") 계열입니다.
- 39행: grid(row=0, column=0...) — 바구니 안에서 왼쪽(0번 칸)에 배치합니다.
- 41~44행: 처음에는 ? 모양을 보여줄 큰 주사위 그림(이미지) 레이블입니다. font 크기가 36으로 아주 큼니다.
- 46~50행: 주사위 눈의 숫자를 텍스트로 보여줄 레이블입니다. (기본값 "숫자: 0")

```
# 플레이어 영역
self.player_frame = tk.LabelFrame(
    self.board_frame,
    text=" 나 (Player) ",
    font=("Helvetica", 12, "bold"),
    bg="ffffff",
    fg="#2b6cb0",
    padx=25,
    pady=20,
    bd=2,
)
self.player_frame.grid(row=0, column=0, padx=20)

self.lbl_p_dice = tk.Label(
    self.player_frame, text=" ? ", font=("Helvetica", 36), bg="ffffff"
) # 글자 크기를 키워 가독성을 높였습니다.
self.lbl_p_dice.pack(pady=5)

self.lbl_p_sum = tk.Label(
    self.player_frame,
    text="숫자: 0",
    font=("Helvetica", 14, "bold"),
    bg="ffffff",
)
self.lbl_p_sum.pack()
```

# 주사위 게임 코드 설명

## 52~75행

플레이어 영역과 완벽히 대칭되는 구조입니다.

- 63행: grid(row=0, column=1...) — 플레이어 바로 오른쪽인 오른쪽(1번 칸)에 배치합니다.
- 57행: AI 구역의 테두리 글자는 빨간색(fg="#c53030") 계열로 차별화를 주었습니다.

```
# AI 영역
self.ai_frame = tk.LabelFrame(
    self.board_frame,
    text=" 🎲 AI ",
    font=("Helvetica", 12, "bold"),
    bg="ffffff",
    fg="#c53030",
    padx=25,
    pady=20,
    bd=2,
)
self.ai_frame.grid(row=0, column=1, padx=20)

self.lbl_ai_dice = tk.Label(
    self.ai_frame, text=" ? ", font=("Helvetica", 36), bg="ffffff"
)
self.lbl_ai_dice.pack(pady=5)

self.lbl_ai_sum = tk.Label(
    self.ai_frame, text="숫자: 0", font=("Helvetica", 14, "bold"), bg="ffffff"
)
self.lbl_ai_sum.pack()
```

## 77~98행

- 78~85행: 중간 결과나 최종 승패("승리!", "패배!")를 안내해 줄 문구 레이블입니다.
- 88~97행: 초록색(bg="#48bb78")의 주사위 굴리기 버튼입니다. 마우스를 올리면 손가락 모양(cursor="hand2")으로 변합니다.
- 94행: command=self.play\_round — 이 버튼을 누르면 아래에 정의된 play\_round 함수가 실행됩니다.

```
# 결과창 영역
self.result_label = tk.Label(
    window,
    text="버튼을 눌러 게임을 시작하세요!",
    font=("Helvetica", 14, "bold"),
    bg="#f0f4f8",
    fg="#4a5568",
)
self.result_label.pack(pady=30)

# 조작 버튼
self.roll_button = tk.Button(
    window,
    text="주사위 굴리기 🎲",
    font=("Helvetica", 14, "bold"),
    bg="#48bb78",
    fg="white",
    padx=20,
    pady=10,
    command=self.play_round,
    activebackground="#38a169",
    activeforeground="white",
    cursor="hand2",
)
self.roll_button.pack(pady=10)
```

# 주사위 게임 코드 설명

## 100~106행

- 100~102행: 호출되면 1부터 6까지의 숫자 중 하나를 무작위로 선택해 돌려줍니다.
- 104~106행: 전달받은 숫자(1~6)에 매칭되는 특수문자 주사위 모양(🎲 ~ 🎲)을 찾아 변환해 줍니다.

```
def roll_dice(self):  
    """1~6 주사위 1개를 굴려 눈을 반환"""  
    return random.randint(1, 6)  
  
def get_dice_emoji(self, num):  
    """숫자에 맞는 주사위 모양 이모지 반환"""  
    emojis = {1: "🎲", 2: "🎲", 3: "🎲", 4: "🎲", 5: "🎲", 6: "🎲"}  
    return emojis.get(num, " ? ")
```

## 108~140행

- 110~111행: 사용자가 버튼을 마구 연타하는 것을 막기 위해, 누르는 즉시 버튼을 잠그고 텍스트를 “굴리는 중...”으로 바꾼 뒤 화면을 새로고침(update)합니다.
- 114~116행: 플레이어의 점수를 뺄고 화면의 주사위 모양과 텍스트를 즉시 변경합니다.
- 119~121행: AI 구역이 바뀌기 전, 안내창에 “AI가 주사위를 굴리는 중...”을 띄우고 0.8초간 프로그램을 잠깐 대기시킵니다. (시각적 효과)
- 124~126행: AI의 점수를 뺄고 AI 구역의 주사위 모양과 텍스트를 바꿉니다.
- 129~137행: if-elif-else 구조로 플레이어 점수와 AI 점수를 비교하여 조건에 맞는 승리/패배/무승부 문구와 글자 색상을 지정합니다.
- 140행: 라운드가 끝났으므로 비활성화했던 버튼을 다시 누를 수 있게 정상(normal) 상태로 되돌립니다.

```
def play_round(self):  
    # 버튼 일시 비활성화 (연타 방지)  
    self.roll_button.config(state="disabled", text="굴리는 중...")  
    self.window.update()  
  
    # 1. 플레이어 주사위 굴리기  
    p_score = self.roll_dice()  
    self.lbl_p_dice.config(text=self.get_dice_emoji(p_score))  
    self.lbl_p_sum.config(text=f"숫자: {p_score}")  
  
    # AI 생각하는 척 딜레이 주기  
    self.result_label.config(text="🤖 AI가 주사위를 굴리는 중...", fg="#718096")  
    self.window.update()  
    time.sleep(0.8) # 1개라 연산이 빠르니 딜레이도 살짝 줄였습니다.  
  
    # 2. AI 주사위 굴리기  
    ai_score = self.roll_dice()  
    self.lbl_ai_dice.config(text=self.get_dice_emoji(ai_score))  
    self.lbl_ai_sum.config(text=f"숫자: {ai_score}")  
  
    # 3. 승리 패배 및 화면 업데이트  
    if p_score > ai_score:  
        self.result_label.config(text="🏆 승리! 당신이 이겼습니다! 🎉", fg="#2f855a")  
    elif p_score < ai_score:  
        self.result_label.config(text="🏆 패배! AI가 이겼습니다.", fg="#c53838")  
    else:  
        self.result_label.config(text="🤝 무승부! 다시 대결해보세요.", fg="#4a5568")  
  
    # 버튼 복구  
    self.roll_button.config(state="normal", text="주사위 굴리기 🎲")
```

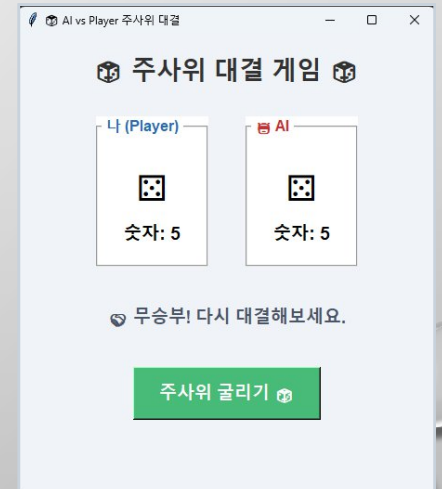
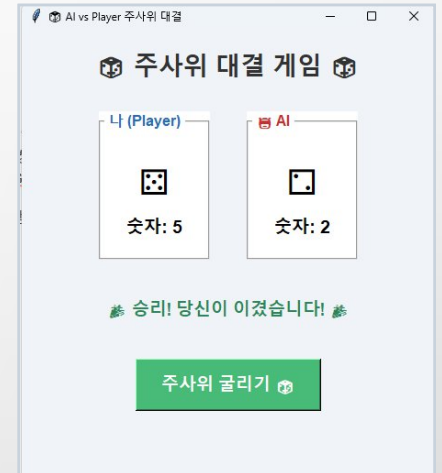


# 주사위 게임 코드 설명

## 144~147행

- 144행: 이 파일이 다른 곳에 서브 파일로 불러와진 게 아니라, 직접 실행되었는지를 확인합니다.
- 145행: 기본이 되는 Tkinter 윈도우 창(root)을 실제로 생성합니다.
- 146행: 우리가 위에서 설계한 DiceGameGUI 설계도에 root 창을 주입하여 게임 인스턴스를 만듭니다.
- 147행: 창이 닫히기 전까지 프로그램이 종료되지 않고 마우스 클릭 등의 이벤트를 계속 감지하며 대기하도록 루프를 돌립니다.

```
# GUI 창 띄우기
if __name__ == "__main__":
    root = tk.Tk()
    game = DiceGameGUI(root)
    root.mainloop()
```



## 느낀점

이번에 프로그래밍을 처음 배우고 , 단순히 컴퓨터 명령어를 외우는 걸 넘어 코드가 돌아가는 원리와 논리적인 문제 해결 방식을 배울 수 있었습니다.

가장 크게 배운 점은 컴퓨터의 정확성과 디버깅의 중요성이었습니다. 컴퓨터는 아주 사소한 오타나 들여쓰기 오류 하나만 있어도 가차 없이 에러를 띄웠습니다. 그러면서 오류를 고치는데 정말 힘들었습니다.

두 번째는 문제를 쪼개서 생각하는 것이었습니다. 프로그램을 만들려면 내가 구현하고 싶은 기능을 최소 단위의 단계로 나누어야 했습니다. 조건문과 반복문을 어떻게 배치하느냐에 따라 데이터가 움직이는 걸 보면서, 복잡한 문제도 순서대로 차근차근 접근하면 해결할 수 있다는 인과관계를 체감했습니다.

결론적으로 이번 준비 과정은 단순히 코딩 기술을 배운 것을 넘어, 문제를 논리적이고 단계적으로 해결하는 컴퓨팅 사고력을 기를 수 있었던 뜻깊은 경험이었습니다.

## 나의 예상 점수

GitHub: 15/15  
프로젝트 발표: 15/15  
총합: 30/30





**Thank you!**

발표를 들어주셔서 감사합니다.

